Retry

Retry:

- In distributed system, call to downstream service might fail because of Transient issues.
- Transient issues means short temporary issues like network issue, timeouts etc.
- And retry of same request after a short delay can get succeed.

A question, might come to you that:

Okay, but still why we should do retry, why can't user or client can invoke our API again, so that we will automatically invoke our downstream again, why we have to retry ourself?

For each call, lot of heavy processing is done and its not free of cost (memory, time, resource etc.). Say 90% of the API functionality completed and then while invoking the downstream, a transient issue comes.

Is it a good idea to dispose off 90% of the task done and with new Api (same request) again perform the same 90% task again.
or
we can do retry and if it get succeed, we will not only save cost but provides better user experience.

Concept && Coding

With above understanding, we know Why RETRY is important, but now Question is:

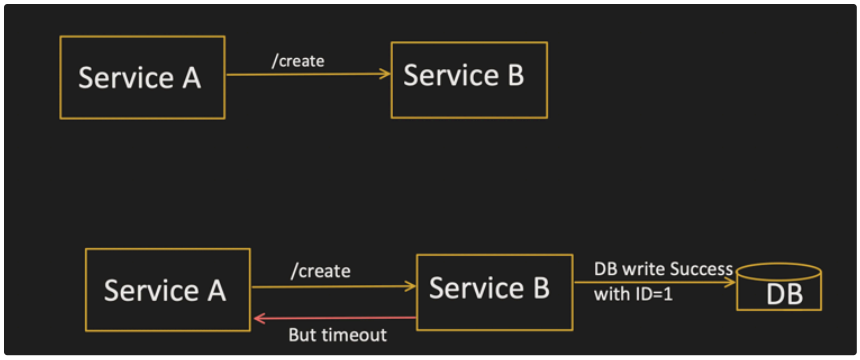
- When to retry?
- How to retry?

<u>When to retry?</u>

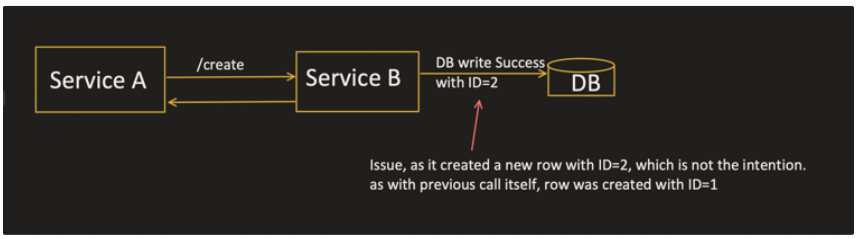
- Do not retry on permanent failures :
  e.g., 4xx errors
  4xx are validation errors, means there is some issue with input request.
  So even you retry with same request, it will fail again in the validation error.

  -Do not retry, if operations is non-idempotent.
  Idempotency make sure that, if request is retried multiple times, duplicate processing will not happen.

For e.g.,



What if user retry in this case, as timeout happens but this endpoint is not idempotent



If there is any doubt with, what is Idempotency and how to design such APIs, I have already covered it in depth in HLD playlist, kindly have a look:

Concept && Coding

Rule of Thumb of this is, Retry on 5xx errors:
5xx error means, there is some issue with system and request is not successful.
It could be a transient issue.
But its pretty safe to retry those errors.

- Retry Network errors and Timeouts only if endpoint ensures Idempotency.

Retry Decision Table

HTTP Status Code	Meaning	Retry?
200 OK	Success	No
400 Bad Request	Client error (invalid input)	No
401 Unauthorized	Authentication needed	No
403 Forbidden	Access denied	No
404 Not Found	Resource not found	No
409 Conflict	Resource not found	No
429 Too Many Requests	Rate limiting	Yes (with Delay)
500 Internal Server Error	Server issue	Yes
502 Bad Gateway	Downstream service failure	Yes
503 Service Unavailable	Service down/overloaded	Yes
Network Error	Connection failure, timeout	Yes (Idempotent only)

Concept && Coding

How to retry?

Types of Retry:-

-Fixed Interval

- Retry after a constant delay
  Ex:

Total attempt = 4 (i.e. original call + 3 retries)

With 2 second fixed Interval delay

original call : 2

1st Retry : 4

2nd Retry : 6

3rd Retry : 8

Advantage of this strategy:

- Very simple.
- Easy to configure and debug.

Disadvantage of this strategy:

- High chance of Retry storm
  (Thundering heard problem)

  Thundering herd problem : Many clients retry **at the same time**, causing a surge of retry traffic → system spends resources on retries instead of handling new requests.

Already covered, Thundering Herd problem and its solution in depth in HLD playlist.
Check it out, if need more info about this issue.

```java
@RestController
@RequestMapping("/orders")
public class OrderController {

    @Autowired
    OrderService orderService;

    @GetMapping("/{id}")
    public void callProductAPI(@PathVariable String id) {
        orderService.invokeProductAPI(id);
    }
}

```

Concept && Coding

```java
@FeignClient(name = "product-service")
public interface ProductClient {

    @GetMapping(value = "/products/{id}")
    String getProductById(@PathVariable("id") String id);
}

```

```java
@Component
public class OrderService {

    @Autowired
    ProductClient productClient;
  /*
    @Retry Applied on top of method, where only downstream service
    invocation is present. Do make sure that, method should
    contains only that code which you want to retry.
    When all retry will finish off, then fallback method will be invoked.
 */

    @Retry(name = "productService", fallbackMethod = "productFallback")
    public void invokeProductAPI(String id) {
        String response = productClient.getProductById(id);
```

```
    }

    public void productFallback(String id, Throwable t) {
        System.out.println("Product Service is busy");
    }
}

```

application.properties

```
server.port=8081
spring.application.name=order-service
eureka.client.service-url.defaultZone=http://localhost:8761/eureka

# fixed delay interval
resilience4j.retry.instances.productService.maxAttempts=3
resilience4j.retry.instances.productService.waitDuration=2s

```

Testing setup:

```
@Component
public class OrderService {

    @Autowired
    ProductClient productClient;

    @Retry(name = "productService", fallbackMethod = "productFallback")
    public void invokeProductAPI(String id) {

        String response = "";
        try {
            response = productClient.getProductById(id);
/*
      Stopped the Product service server, so that failure happens at
      the time of invocation, and in this method added try catch
      block with System.out.println statement for testing.
*/
        } catch (Exception e) {
            System.out.println("not able to invoke Product");
            throw e;
        }

        System.out.println("Response from product api call: " + response);
    }

    public void productFallback(String id, Throwable t) {
        System.out.println("Product Service is busy");
    }
}

```

Concept && Coding

Output:

Max attempt is 3 (means 1 original call and 2 retry call)

```
2025-07-17T20:56:10.282+05:30  INFO 86027 --- [order-service] [nio-8081-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring DispatcherServlet '
2025-07-17T20:56:10.282+05:30  INFO 86027 --- [order-service] [nio-8081-exec-1] o.s.web.servlet.DispatcherServlet        : Initializing Servlet 'dispatcherServlet
2025-07-17T20:56:10.283+05:30  INFO 86027 --- [order-service] [nio-8081-exec-1] o.s.web.servlet.DispatcherServlet        : Completed initialization in 1 ms
2025-07-17T20:56:10.333+05:30  WARN 86027 --- [order-service] [nio-8081-exec-1] o.s.c.l.core.RoundRobinLoadBalancer      : No servers available for service: produ
2025-07-17T20:56:10.334+05:30  WARN 86027 --- [order-service] [nio-8081-exec-1] .s.c.o.l.FeignBlockingLoadBalancerClient : Load balancer does not contain an instan
not able to invoke Product        ---> Original call at 20:56:10
2025-07-17T20:56:12.343+05:30  WARN 86027 --- [order-service] [nio-8081-exec-1] o.s.c.l.core.RoundRobinLoadBalancer      : No servers available for service: produ
2025-07-17T20:56:12.344+05:30  WARN 86027 --- [order-service] [nio-8081-exec-1] .s.c.o.l.FeignBlockingLoadBalancerClient : Load balancer does not contain an instan
not able to invoke Product        ---> 1st retry call at 20:56:12 = 2s delay
2025-07-17T20:56:14.346+05:30  WARN 86027 --- [order-service] [nio-8081-exec-1] o.s.c.l.core.RoundRobinLoadBalancer      : No servers available for service: produ
2025-07-17T20:56:14.346+05:30  WARN 86027 --- [order-service] [nio-8081-exec-1] .s.c.o.l.FeignBlockingLoadBalancerClient : Load balancer does not contain an instan
not able to invoke Product        ---> 2nd retry call at 20:56:14 = 2s delay
Product Service is busy           ---> All attempt finished, so executed fallback method
```

Internally what might have happened:

As we know, AOP generated the proxy code. Lets see below, how it might have looked like:

```java
public Retry fixedRetry() {
    RetryConfig config = RetryConfig.custom()
            .maxAttempts(4)
      /*
        IntervalFunction , helps to determine the delay between retries.
        It has many pre-defined functions like:
       - IntervalFunction of(long intervalMillis)  //for fixed Delay
      - IntervalFunction ofExponentialBackoff(long initialIntervalMillis,
        double multiplier, long maxIntervalMillis)
          etc..
      */
            .intervalFunction(IntervalFunction.of(2000))
      /*
      Retry will only happen for these or they sub class exceptions.
      */
            .retryExceptions(IOException.class, ArithmeticException.class)
            .build();
/*
  Retry will only happen for these or they sub class exceptions.
*/
    return Retry.of("fixedRetry", config);
}

```

Concept && Coding

<u>Exponential Backoff :</u>

- Delay increases exponentially.

  delay = Base * 2no of failed retry attempt

Ex:

Total attempt = 4 (i.e. original call + 3 retries)

With Base = 100ms

Original call

Delay for 1st Retry : 100 ∗ 2^0 = 100 ms

Delay for 2nd Retry : 100 ∗ 2^1 = 200 ms

Delay for 3rd Retry : 100 ∗ 2^2 = 400 ms

Advantage of this strategy:

- Reduce load on downstream service during outage.
- Helps to avoid Thundering Herd problem to some extent.

Disadvantage of this strategy:

- If outage period is short, user might wait longer than needed.
- Still some risk of Thundering Herd problem, because all clients use the same formula to calculate retry interval (Base * 2 ^ retry attempt)

All same only configuration changes.

application.properties

```
server.port=8081

spring.application.name=order-service

eureka.client.service-url.defaultZone=http://localhost:8761/eureka

#exponential backoff
resilience4j.retry.instances.productService.maxAttempts=4
resilience4j.retry.instances.productService.waitDuration=1s
resilience4j.retry.instances.productService.enableExponentialBackoff=true
resilience4j.retry.instances.productService.exponentialBackoffMultiplier=2

```

Concept && Coding

Backoff.java (framework class)

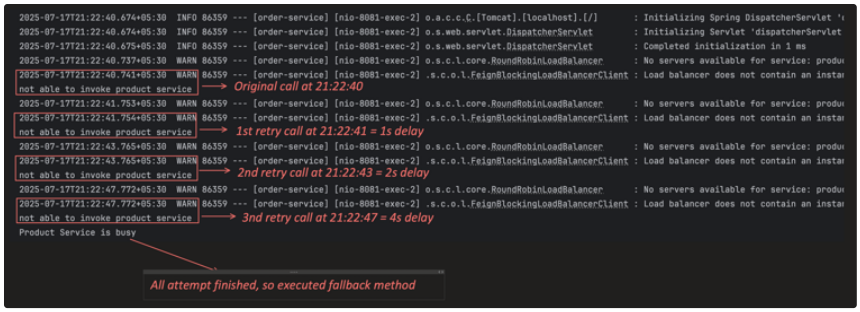
```
try {
  /*
  Formula is:
  initial wait time * (factor ^ no of failed retry attempts)

  1st retry = 1000ms * (2 ^ 0) = 1000ms = 1sec delay
  2nd retry = 1000ms * (2^1) = 2000ms = 2sec delay
  3rd retry = 1000 * (2^2) = 4000ms = 4sec delay
  */
    nextBackoff = firstBackoff.multipliedBy((long) Math.pow(factor, (context.iteration() - 1)));
    if (nextBackoff.compareTo(maxBackoffInterval) >= 0) {
        nextBackoff = maxBackoffInterval;
    }
}

```

Output:

Max attempt is 4 (means 1 original call and 3 retry call)



AOP generated the proxy code. might have looked like:

```
public Retry exponentialJitterRetry() {

    RetryConfig config = RetryConfig.custom()
        .maxAttempts(6)
        .intervalFunction(
          /*
          Using appropriate IntervalFunction method for delay computation
      */
            IntervalFunction.ofExponentialRandomBackoff(
                initialIntervalMillis: 2000,
                multiplier: 2
            )
        )
        .retryExceptions(Exception.class)
        .build();

    return Retry.of(name: "exponentialBackOffRetry", config);
}

```

Concept && Coding

Exponential Backoff + Jitter :

Similar to Exponential Backoff, but adds Jitter/randomness to delay to avoid retry storms.

Exponential Backoff + Jitter

delay = random_between (0, min(maxDelay, baseDelay * 2^ failed retry attempt))

Advantage of this strategy:

- Best for distributed system, as Thundering herd issuer probability becomes very less.
- Spread retries over time, reducing peak load.

Disadvantage of this strategy:

- Not easy to debug and little complex.

```
server.port=8081
spring.application.name=order-service
eureka.client.service-url.defaultZone=http://localhost:8761/eureka

# exponential backoff + jitter
resilience4j.retry.instances.productService.maxAttempts=4
resilience4j.retry.instances.productService.waitDuration=1s
```

```
resilience4j.retry.instances.productService.enableExponentialBackoff=true
resilience4j.retry.instances.productService.exponentialBackoffMultiplier=2
resilience4j.retry.instances.productService.enableRandomizedWait=true

```

Custom Retry

Define your own retry delay strategy.

Advantage of this strategy:

- Full control and flexibility

Disadvantage of this strategy:

- Harder to maintain.

```java
@Configuration
public class Config {

    @Bean
    public Retry customRetry() {

      /*
      IntervalFunction, defines the wait time between retries.
      */
        IntervalFunction fibonacciInterval = attempt -> {
            return 2000L; // return milliseconds as long
        };

       /*
        Within RetryConfig object, we provide all the configs like max attempt, which IntervalFunction to use to compute the delay etc.
       */
        RetryConfig config = RetryConfig.custom()
                .maxAttempts(6)
                .intervalFunction(fibonacciInterval)
                .retryExceptions(Exception.class)
                .build();

        /*
          Then create Retry object from RetryConfig.
          We can give any name to our Retry.
        */
        return Retry.of("customRetry", config);
    }
}

```

Concept && Coding

```java
@Component
public class OrderService {

    @Autowired
    ProductClient productClient;

    @Autowired
    Retry customRetry;

    public void invokeProductAPI(String id) {
        try {
```

```java
            /*
            Need to manually wrap an object within Retry.
            As @Retry annotation works only for those Retry
            types for which we can provide config in application.properties.
           */
                customRetry.executeSupplier(() -> {
                    System.out.println("calling product service at " + java.time.LocalTime.now());
                    return productClient.getProductById(id);
                });
            } catch (Exception e) {
                System.out.println("all retries failed, this is fallback");
            }
        }
}

```

Output:

```
2025-07-17T22:35:45.732+05:30  INFO 87052 --- [order-service] [nio-8081-exec-2]
2025-07-17T22:35:45.732+05:30  INFO 87052 --- [order-service] [nio-8081-exec-2]
calling product service at 22:35:45.759176  ——→  Original attempt at 22:35:45
2025-07-17T22:35:45.792+05:30  WARN 87052 --- [order-service] [nio-8081-exec-2]
2025-07-17T22:35:45.793+05:30  WARN 87052 --- [order-service] [nio-8081-exec-2]
calling product service at 22:35:47.800728  ——→  1st retry attempt at 22:35:47 (2sec delay)
2025-07-17T22:35:47.802+05:30  WARN 87052 --- [order-service] [nio-8081-exec-2]
2025-07-17T22:35:47.803+05:30  WARN 87052 --- [order-service] [nio-8081-exec-2]
calling product service at 22:35:49.809078  ——→  2nd retry attempt at 22:35:49 (2sec delay)
2025-07-17T22:35:49.809+05:30  WARN 87052 --- [order-service] [nio-8081-exec-2]
2025-07-17T22:35:49.809+05:30  WARN 87052 --- [order-service] [nio-8081-exec-2]
calling product service at 22:35:51.814954  ——→  3rd retry attempt at 22:35:51 (2sec delay)
2025-07-17T22:35:51.816+05:30  WARN 87052 --- [order-service] [nio-8081-exec-2]
2025-07-17T22:35:51.816+05:30  WARN 87052 --- [order-service] [nio-8081-exec-2]
calling product service at 22:35:53.821018  ——→  4st retry attempt at 22:35:53 (2sec delay)
2025-07-17T22:35:53.823+05:30  WARN 87052 --- [order-service] [nio-8081-exec-2]
2025-07-17T22:35:53.823+05:30  WARN 87052 --- [order-service] [nio-8081-exec-2]
calling product service at 22:35:55.829334  ——→  5th retry attempt at 22:35:55 (2sec delay)
2025-07-17T22:35:55.831+05:30  WARN 87052 --- [order-service] [nio-8081-exec-2]
2025-07-17T22:35:55.831+05:30  WARN 87052 --- [order-service] [nio-8081-exec-2]
all retries failed, this is fallback  ——→  All attempts finished to fallback invoked
```

oncept && Coding